



---

## **Práctica 1A**

### **K-means en los pixeles de una imagen**

Aprendizaje No Supervisado y Visualización  
Unidad 1 · Maestría en Ciencia de Datos e Información

---

Bryan Rodríguez Abarca

Matrícula 26DF1185A9

14 de agosto de 2026

La práctica consiste en tratar cada pixel de una fotografía como un dato de tres números (sus canales rojo, verde y azul), agrupar esos 205,360 datos en 8 grupos con k-means, y volver a pintar la fotografía usando únicamente los 8 colores representativos que el algoritmo encuentre.

Todos los números que aparecen en el texto salen de ejecutar este mismo notebook con la semilla que se indica más abajo.

## 1. Qué es k-means y qué significa aplicarlo a pixeles

K-means es un algoritmo de agrupamiento: recibe un montón de datos sin etiqueta y los reparte en  $k$  grupos, de modo que los datos de un mismo grupo estén cerca entre sí. “Cerca” se mide con la distancia usual (euclidiana). Cada grupo queda representado por un punto llamado **centroide**, que es el promedio de todos los datos que cayeron en él. El algoritmo alterna dos pasos hasta que deja de haber cambios: asignar cada dato a su centroide más cercano, y recalcular cada centroide como el promedio de sus datos.

Aplicado a una imagen, el truco está en qué se considera un “dato”. Aquí un dato es **un pixel**, descrito por tres números entre 0 y 255: cuánto rojo, cuánto verde y cuánto azul tiene. Como la distancia entre dos pixeles se calcula sobre esos tres números, dos pixeles están “cerca” cuando su color se parece. Por lo tanto **agrupar pixeles por distancia es agruparlos por color** — contando el brillo: para esta distancia un verde claro y un verde oscuro quedan lejos aunque sean el mismo tono —, y los 8 centroides que salgan son los 8 colores promedio que mejor resumen la fotografía: su paleta.

La posición del pixel dentro de la foto no interviene en ningún momento. Dos hojas en esquinas opuestas de la imagen caen en el mismo grupo si comparten el tono; el algoritmo no sabe que existe la noción de “vecino”.

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import sklearn, matplotlib, PIL, sys

print("python      ", sys.version.split()[0])
print("numpy       ", np.__version__)
print("scikit-learn", sklearn.__version__)
print("matplotlib  ", matplotlib.__version__)
print("Pillow      ", PIL.__version__)
```

```
python      3.9.6
numpy       2.0.2
scikit-learn 1.6.1
matplotlib  3.9.4
Pillow      11.3.0
```

## 2. Los datos: de una fotografía a una tabla de tres columnas

Se carga la imagen y se muestra tal cual, para tener la referencia visual contra la que comparar el resultado final.

```
img = Image.open("image.jpg")
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.axis("off")
plt.title("Imagen original")
plt.show()
```

Imagen original



```
img = np.array(img)
s = img.shape
print("Tamaño de la imagen:", s)

X = img.reshape(s[0] * s[1], s[2])
print("Tamaño de los datos:", X.shape)
print("Primeros 3 pixeles:\n", X[:3])
print("Colores distintos en la foto:", len(np.unique(X, axis=0)))
```

Tamaño de la imagen: (340, 604, 3)

Tamaño de los datos: (205360, 3)

Primeros 3 pixeles:

```
[[ 77  89  49]
```

```
[ 85  97  61]
[118 129  99]]
Colores distintos en la foto: 71747
```

La imagen llega como un arreglo de forma (340, 604, 3): 340 renglones de píxeles, 604 columnas, y 3 números por píxel. K-means, en cambio, espera una **tabla de dos dimensiones**: un renglón por dato y una columna por característica. Por eso se usa `reshape(340*604, 3)`, que produce la tabla (205360, 3).

`reshape` no altera ni un solo número: los  $205,360 \times 3$  valores siguen siendo los mismos y en el mismo orden de lectura; lo único que cambia es cómo están agrupados. Se pasa de “una rejilla de  $340 \times 604$  casillas, cada una con tres números” a “una lista de 205,360 renglones de tres números”. Esa reversibilidad es lo que permitirá, al final, volver a acomodar el resultado en forma de foto.

El conteo de colores distintos muestra el tamaño real del problema: la foto usa **71,747 colores diferentes**, y el objetivo es reducirlos a 8.

### 3. El agrupamiento con $k = 8$

`fit_predict(X)` entrena el modelo (encuentra los 8 centroides) y devuelve en la misma llamada la etiqueta de grupo de cada dato. El resultado se guarda en `X_km`, que es entonces un arreglo de 205,360 números enteros, cada uno entre 0 y 7: `X_km[i]` es el número de grupo al que pertenece el píxel `i`. No contiene colores, contiene números de grupo.

Los centroides se toman del atributo `cluster_centers_` y se guardan en `centroids`: un arreglo de 8 renglones por 3 columnas, con los valores RGB promedio de cada grupo.

#### 3.1. Decisión personal 1 — la semilla

K-means empieza eligiendo los centroides iniciales al azar, así que dos corridas del mismo código pueden terminar en soluciones distintas. Para que este documento sea reproducible fijo `random_state=1185`, tomado de mi matrícula (26DF1185A9). Fijar la semilla no vuelve “correcta” a la solución: solo garantiza que quien ejecute este notebook obtenga exactamente los números que aquí se reportan. En la sección 6 comparo esta semilla contra `random_state=0` para mostrar cuánto cambia el resultado.

```
km = KMeans(n_clusters=8, random_state=1185)
X_km = km.fit_predict(X)           # etiqueta de grupo de cada píxel
centroids = km.cluster_centers_   # los 8 colores representativos

print("X_km      :", X_km.shape, X_km.dtype)
print("etiquetas:", np.unique(X_km))
print("centroids:", centroids.shape, centroids.dtype)
print("inercia   :", km.inertia_)
print("iteraciones:", km.n_iter_)
```

```

X_km      : (205360,) int32
etiquetas: [0 1 2 3 4 5 6 7]
centroids: (8, 3) float64
inercia    : 92828730.63494289
iteraciones: 50

```

La **inercia** es la suma de las distancias al cuadrado de cada pixel a su propio centroide: mide qué tan apretados quedaron los grupos, y es justamente la cantidad que k-means intenta hacer pequeña. Sirve para comparar dos corridas con el mismo  $k$ , no para decidir cuál  $k$  es mejor: al subir  $k$  suele salir más baja, como se ve en la sección 7.

Un detalle que me ayudó a no equivocarme fueron los tipos de dato: **X\_km** es de números enteros porque son identificadores de grupo, mientras que **centroids** es de decimales porque son promedios. Un promedio de valores enteros casi nunca cae en un entero, y eso obligará a convertir a **uint8** antes de dibujar.

## 4. Los centroides y la paleta

Se imprimen los 8 centroides con tres datos por renglón: sus valores RGB tal cual los devolvió el algoritmo, su código hexadecimal (el formato con el que se nombran los colores en la web) y el porcentaje de pixeles de la foto que quedó en ese grupo. Ese porcentaje sale de contar cuántas veces aparece cada etiqueta en **X\_km**.

```

conteo = np.bincount(X_km, minlength=8)

total = len(X_km)

print(" #      R      G      B      hex      pixeles      %")
for i, c in enumerate(centroids):
    r, g, b = c.astype("uint8")
    hexa = f"#{r:02X}{g:02X}{b:02X}"
    print(f" {i}   {c[0]:6.2f}  {c[1]:6.2f}  {c[2]:6.2f}  {hexa}"
          f"   {conteo[i]:8d}  {100*conteo[i]/total:6.2f}%")
print()
print(f"suma: {conteo.sum()} pixeles ({100*conteo.sum()/total:.2f}%)")

```

| # | R      | G      | B      | hex     | pixeles | %     |
|---|--------|--------|--------|---------|---------|-------|
| 0 | 218.12 | 220.00 | 195.80 | #DADBC3 | 5096    | 2.48  |
| 1 | 84.55  | 84.98  | 46.07  | #54542E | 36644   | 17.84 |
| 2 | 28.63  | 30.52  | 7.37   | #1C1E07 | 23093   | 11.25 |
| 3 | 134.99 | 131.90 | 86.66  | #868356 | 30045   | 14.63 |
| 4 | 58.32  | 60.20  | 26.17  | #3A3C1A | 33265   | 16.20 |
| 5 | 182.26 | 182.01 | 145.10 | #B6B691 | 16808   | 8.18  |
| 6 | 157.53 | 156.19 | 114.83 | #9D9C72 | 25092   | 12.22 |
| 7 | 109.94 | 108.02 | 64.84  | #6D6C40 | 35317   | 17.20 |

```
suma: 205360 pixeles (100.00 %)
```

```
palette = np.zeros((50, 50 * len(centroids), 3))
for i, c in enumerate(centroids):
    color = c[:3].astype("uint8")
    palette[:, i*50: i*50+50, :] += color
plt.imshow(palette.astype("uint8"))
plt.axis("off")
plt.title("Paleta: los 8 centroides en el orden que los devolvió KMeans")
plt.show()
```

Paleta: los 8 centroides en el orden que los devolvió KMeans



#### 4.1. Decisión personal 2 — la paleta ordenada por peso

El orden en que scikit-learn entrega los centroides es arbitrario: depende de dónde cayeron los centros iniciales, no de ninguna propiedad de la foto. Repito entonces la paleta ordenando los colores de mayor a menor cantidad de pixeles, que sí es una propiedad de la imagen: leída así, la barra dice de un vistazo de qué color es la fotografía.

```
orden = np.argsort(-conteo) # de más pixeles a menos
centroids_ord = centroids[orden]

paleta_ord = np.zeros((50, 50 * len(centroids_ord), 3))
for i, c in enumerate(centroids_ord):
    paleta_ord[:, i*50: i*50+50, :] += c[:3].astype("uint8")

plt.figure(figsize=(8, 2))
plt.imshow(paleta_ord.astype("uint8"))
plt.axis("off")
plt.title("Paleta ordenada por número de pixeles (de mayor a menor)")
plt.show()

for i in orden:
    r, g, b = centroids[i].astype("uint8")
    pct = 100 * conteo[i] / len(X_km)
    print(f"grupo {i}: #{r:02X}{g:02X}{b:02X} {pct:5.2f}%")
```

Paleta ordenada por número de pixeles (de mayor a menor)



```
grupo 1: #54542E 17.84 %
grupo 7: #6D6C40 17.20 %
grupo 4: #3A3C1A 16.20 %
grupo 3: #868356 14.63 %
grupo 6: #9D9C72 12.22 %
grupo 2: #1C1E07 11.25 %
grupo 5: #B6B691 8.18 %
grupo 0: #DADBC3 2.48 %
```

## 4.2. Decisión personal 3 — qué dice esta paleta de la fotografía

La foto es un bosque con neblina: troncos delgados, follaje verde apagado y pasto seco iluminado en el primer plano. Lo que leo en la paleta:

- El grupo más oscuro, cercano a `#1C1E07` (~11 % de los píxeles), no es un color “real” de nada en particular: son los troncos, sus sombras y los huecos entre el follaje. Es el color de la ausencia de luz en esta escena, y por eso es tan verde-negro y no gris.
- Los cinco grupos intermedios (de `#3A3C1A` a `#9D9C72`) concentran el 78 % de la imagen y son casi el mismo color con distinto brillo. Casi todo en la foto es la misma mezcla verde-oliva, y lo que distingue una zona de otra es cuánta luz recibe; por eso k-means, que solo mide distancia, gasta la mayoría de sus grupos en repartir niveles de iluminación y no colores distintos.
- El grupo más claro, alrededor de `#DADBC3`, es el más pequeño (~2.5 %) y sin embargo es el que hace que la foto se lea como “con neblina”: es la luz difusa que se cuela entre los árboles al fondo. Aunque es el grupo más chico, sin él la foto no se vería neblinosa.

También dice algo lo que falta: en la paleta no hay ningún azul ni ningún rojo francos. El follaje y la neblina apagan todo hacia el mismo eje verde.

## 5. Recolorear la imagen

El paso final sustituye cada píxel por el centroide de su grupo. En numpy eso se escribe en una sola operación: `centroids[X_km]`.

La manera en que me lo explico: `centroids[X_km]` significa “por cada etiqueta de `X_km`, trae el color del centroide que lleva ese número”. Como `X_km` tiene 205,360 etiquetas, el resultado tiene 205,360 renglones de 3 números: la foto entera, ya con solo 8 colores distintos, pero todavía en forma de lista. El `reshape(s)` la devuelve a la rejilla de  $340 \times 604 \times 3$  y la vuelve dibujable.

El orden importa: es `centroids[X_km]` y no `X_km[centroids]`. Los índices tienen que ser enteros, y `centroids` son decimales.

```
img_km = centroids[X_km].reshape(s)

print("img_km :", img_km.shape, img_km.dtype)
n_colores = len(np.unique(img_km.reshape(-1, 3), axis=0))
print("colores distintos en img_km:", n_colores)

plt.figure(figsize=(8, 8))
plt.imshow((img_km.astype("uint8")))
```



```
plt.axis("off")
plt.title("Imagen recoloreada con los 8 centroides")
plt.show()
```

```
img_km : (340, 604, 3) float64
colores distintos en img_km: 8
```

Imagen recoloreada con los 8 centroides



Sin el `.astype("uint8")` la imagen me salía blanca por completo, con una advertencia de *clipping*. La causa: `img_km` contiene los centroides, que son decimales, y `imshow` interpreta un arreglo de decimales como valores entre 0.0 y 1.0 — aquí hasta el valor más chico de la tabla (7.37) rebasa ese rango, así que todo se recorta a blanco. Al convertir a entero de 8 bits (`uint8`, de 0 a 255) los números vuelven a significar lo que deben.

Comparando esta imagen con la original: la silueta del bosque, la dirección de la luz y la sensación de neblina se conservan por completo. Se pierden las transiciones suaves, que ahora aparecen como manchas de color plano — sobre todo en el pasto del primer plano, donde antes había decenas de tonos y ahora hay dos o tres — y, más llamativo, **el color**: la versión de 8 colores se ve casi monocroma, en tonos oliva, porque los amarillos dorados del pasto y los verdes vivos del follaje desaparecieron. La explicación está en la paleta de la sección anterior: los ocho centroides son casi el mismo verde-oliva a distinto brillo. Y no es que los pixeles amarillentos sean pocos: es que en esta distancia quedan muy cerca de los verdes-oliva, y al algoritmo le reduce más la inercia dedicar sus grupos a separar niveles de brillo que a separar ese amarillo.



## 6. Comprobación: qué tanto depende el resultado de la semilla

Vuelvo sobre la Decisión 1. Antes de fijar la semilla corrí el mismo código un par de veces y los números no coincidían: la inercia cambiaba y los colores salían en otro orden. Buscando la causa encontré en la documentación de scikit-learn que la inicialización es aleatoria y que en esta versión, por omisión, el algoritmo se corre **una sola vez** (`n_init="auto"` con el arranque `k-means++`), así que nada descarta una corrida con mala suerte.

Para medirlo comparo mi semilla contra `random_state=0`. Las paletas no se pueden comparar tal cual porque los grupos salen en orden arbitrario; las ordené por brillo (la suma de los tres canales) para emparejar cada color con su equivalente.

```
km0 = KMeans(n_clusters=8, random_state=0).fit(X)

print(f"inercia rs=1185: {km.inertia_:.2f} ({km.n_iter_} iteraciones)")
print(f"inercia rs=0    : {km0.inertia_:.2f} ({km0.n_iter_} iteraciones)")
print(f"la primera es {100*(km.inertia_/km0.inertia_ - 1):.2f}% mayor")

a = centroids[np.argsort(centroids.sum(axis=1))]
b = km0.cluster_centers_[np.argsort(km0.cluster_centers_.sum(axis=1))]
d = np.linalg.norm(a - b, axis=1)
print("\ndistancia entre centroides emparejados por brillo:")
print(d.round(2))
print(f"máxima: {d.max():.2f}    promedio: {d.mean():.2f}")
print("(los canales van de 0 a 255)")
```

```
inercia rs=1185: 92828730.63 (50 iteraciones)
inercia rs=0    : 91338052.52 (27 iteraciones)
la primera es 1.63 % mayor

distancia entre centroides emparejados por brillo:
[ 1.59  3.6   6.39 15.09 37.44 20.02 14.55 12.11]
máxima: 37.44    promedio: 13.85
(los canales van de 0 a 255)
```

Mi semilla salió perdiendo: la inercia con `random_state=1185` queda 1.63% por encima de la de `random_state=0`, y una de las parejas de centroides difiere en 37.44 unidades de color, una diferencia visible. k-means se quedó en un **óptimo local**: una solución de la que el algoritmo ya no salió, pero peor que otra que sí existe.

De aquí me llevo que reportar la semilla no es un formalismo: sin ella estos números no se pueden reproducir ni comparar con los de nadie más. Y que si el objetivo fuera la mejor paleta y no ilustrar el algoritmo, usaría `KMeans(n_clusters=8, n_init=10)` para que scikit-learn intente diez inicializaciones y se quede con la de menor inercia; reduce el riesgo de la mala suerte, aunque tampoco garantiza encontrar la mejor de todas.

## 7. Comparación con otros valores de k

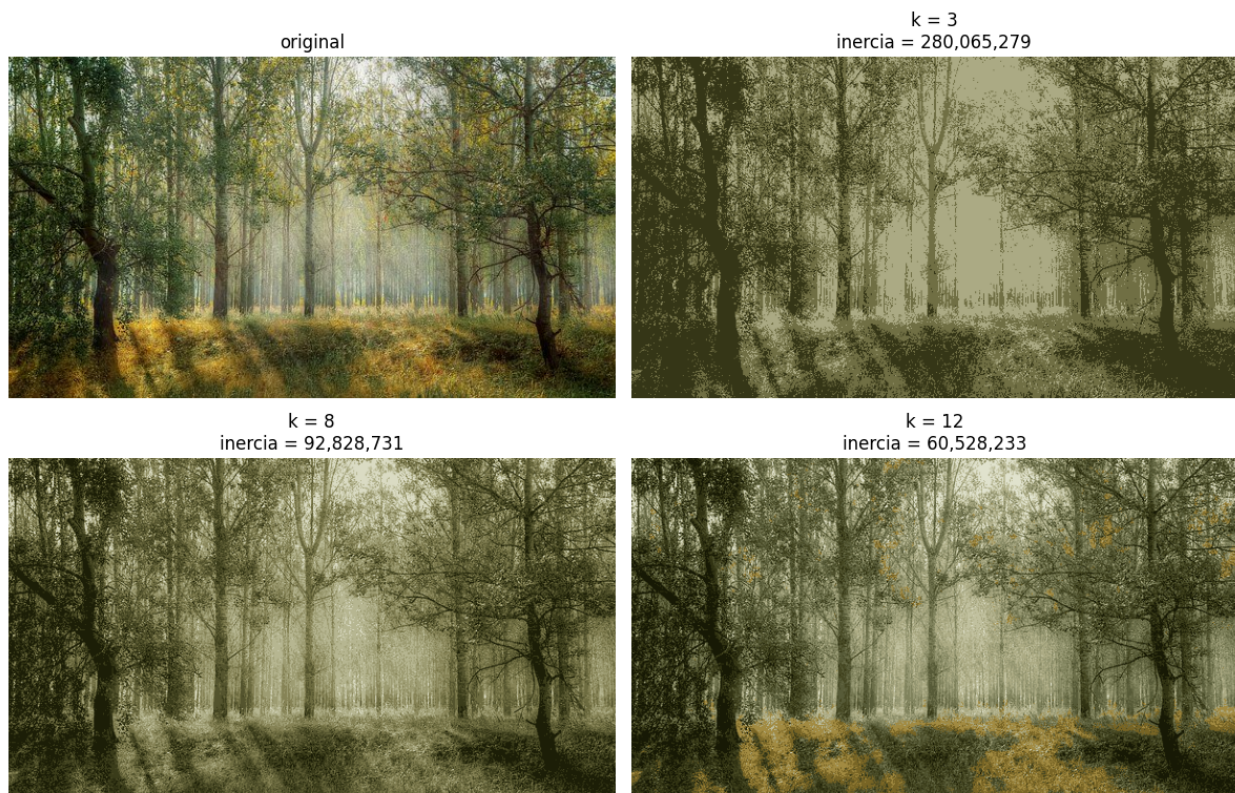
El enunciado pide  $k = 8$ ; esta comparación es un extra propio. Repito el proceso completo con  $k = 3, 8$  y  $12$  para ver qué se gana y qué se pierde al mover el número de grupos.

```
ks = [3, 8, 12]
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.ravel()
axes[0].imshow(img); axes[0].set_title("original"); axes[0].axis("off")

for ax, k in zip(axes[1:], ks):
    m = KMeans(n_clusters=k, random_state=1185).fit(X)
    rec = m.cluster_centers_[m.labels_].reshape(s)
    ax.imshow(rec.astype("uint8"))
    ax.set_title(f"k = {k}\ninercia = {m.inertia_:.0f}")
    ax.axis("off")
    print(f"k = {k:2d}    inercia = {m.inertia_:15,.2f}")

plt.tight_layout()
plt.show()
```

```
k = 3    inercia = 280,065,279.27
k = 8    inercia = 92,828,730.63
k = 12   inercia = 60,528,232.82
```



La inercia bajó en cada paso, y era de esperarse: con más centroides cada pixel tiene uno más cerca. Por eso la inercia por sí sola no sirve para elegir  $k$  — llevada al extremo,  $k = 205,360$  daría inercia cero y la “paleta” sería la foto entera.

El argumento visual es más honesto. Con  $k = 3$  la fotografía se reduce a tres niveles de luz: sombra, tono medio y neblina. El pasto del primer plano se funde con las sombras de los troncos y el bosque pierde toda su profundidad. Con  $k = 8$  vuelven el volumen de los árboles, las franjas de luz sobre el pasto y la neblina del fondo. Con  $k = 12$  se recupera algo de textura en el primer plano, pero el salto es mucho menor que el de 3 a 8. Ese rendimiento decreciente es la razón de que 8 sea una elección razonable para esta imagen.

## 8. Conclusión

Partiendo de una fotografía con **71,747 colores distintos**, k-means con  $k = 8$  encontró ocho colores promedio capaces de reconstruirla de forma reconocible. La imagen recoloreada conserva la composición, la iluminación y la atmósfera de neblina; pierde los degradados suaves, el detalle fino y buena parte de la variedad de color, porque los ocho grupos se repartieron sobre todo niveles de brillo de un mismo verde-oliva.

Lo que hace útil este ejercicio fuera de la clase es que el mismo procedimiento sirve para más de una cosa según qué se guarde al final:

- **Compresión / cuantización de color.** Si se guardan las 8 entradas de la paleta y una etiqueta por pixel, cada pixel pasa de necesitar 3 bytes a necesitar 3 bits (8 valores posibles), sin cambiar el tamaño en pixeles de la imagen. Es la idea detrás de los formatos de imagen con paleta indexada, como GIF.
- **Extracción de paleta.** Los centroides son directamente la identidad cromática de una imagen: es como se generan automáticamente los colores de acento de una portada de disco o de un tema visual a partir de una foto.
- **Segmentación.**  $X_{km}$  reacomodado a la forma de la imagen es un mapa de regiones: qué zonas comparten tono. Es un primer paso barato para separar objetos del fondo cuando el color los distingue.

La limitación principal quedó a la vista en la sección 6: k-means depende de su inicialización y puede detenerse en un óptimo local. Con una sola corrida, como hace scikit-learn por omisión en esta versión, el resultado que se obtiene es *una* buena solución, no *la* mejor.